47f318ff-fa3d-4510-8c11-24f34d2fef4e

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'hecatehq/hecate' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/hecate/hecate (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-11T14:08:36.268+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
fix(adapters): rewrite Claude Code auth error + one-click setup deep-link
</pr_title>

<pr_description>
What an operator sees today

When Claude Code can't sign in, the chat shows:

> Claude Code needs adapter-visible authentication: set `CLAUDE_CODE_OAUTH_TOKEN` from `claude setup-token`, `ANTHROPIC_API_KEY`, or `ANTHROPIC_AUTH_TOKEN` for the adapter environment. Restart Hecate after changing environment credentials.

Five things wrong with that for a non-developer:

1. **Jargon** — "adapter-visible authentication", "adapter environment" mean nothing without context.
2. **Three options at the same priority** — paralyzing, no recommendation.
3. **"Restart Hecate"** as the headline, but the easiest fix doesn't require a restart.
4. **No mention** that Hecate already has a guided setup card in `Settings → External agents` that takes the token via paste (Hecate stores it, injects it into the adapter, no restart).
5. **Conflates two different keys** — operators who configured Anthropic in the Providers tab reasonably think Claude Code "just works." It doesn't, because Claude Code is a separate subprocess with its own credentials. The error message gives them no hint that this is a different surface.

What this PR changes

Server-side (Tier 1) — friendly message + stable marker:

```
Claude Code isn't signed in. This is a separate credential from the Anthropic key in the Providers tab — Claude Code runs as its own program. Click the button below to paste a token from `claude setup-token` into the guided setup card. No restart needed. (claude_code_auth_required)
```

The trailing `(claude_code_auth_required)` is a load-bearing token the chat UI pattern-matches on. It's stripped from the visible copy before rendering, so the operator sees clean prose. Documented at the call site in `internal/agentadapters/auth_status.go`.

Dashboard hints (`auth_status.go`) and probe-side hints (`probe.go`) get the same treatment: lead with the guided setup card, drop the three-env-var enumeration. The card itself documents the env-var alternative for power users.

Client-side (Tier 2) — inline recovery button:

The chat's failed-run notice now renders an **"Open Claude Code setup"** button when the failure carries the `claude_code_auth_required` marker and the message's adapter is `claude_code`. Click:

- Sets `localStorage.hecate.settingsTab = "external_agents"` so SettingsView mounts on the right sub-tab.
- Sets `sessionStorage.hecate.settingsFocus = "claude-code-guided-setup"` so the External Agents tab scrolls-and-flashes the guided setup card on mount.
- Calls `onNavigate("settings")` to switch workspaces.

One click from "chat failed" to "setup card focused, ready to paste."

The flash animation lives in `styles.css` (2.2s teal ring pulse) next to the design tokens.

Verification

- `bun run typecheck` — clean.
- `bun run test` — 707/707 (was 704; +3 new tests on `TranscriptMessageRow`: marker stripping, button rendering + onClick, negation for cancelled runs).
- `go test ./internal/agentadapters/... -count=1` — 213/213.

Risk

- **Marker drift.** If the server-side string is reworded and the `(claude_code_auth_required)` token is accidentally dropped, the UI button silently disappears. Mitigated by an explicit test in `registry_test.go` that asserts the token is present, plus comments at both ends of the contract.
- **Storage unavailable.** `localStorage` / `sessionStorage` access wrapped in try/catch — navigation still works without auto-scroll if storage is denied (private-browsing context, Tauri webview restrictions).
- **Other adapters with similar issues.** The marker convention `(<adapter>_auth_required)` is forward-compatible — Codex / Cursor could use the same pattern. Not implemented in this PR; Claude Code was the reported one.

</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>

<author>@copilot-pull-request-reviewer</author>
Pull request overview

This PR improves the Claude Code external-agent authentication recovery flow by replacing the current jargon-heavy auth error with clearer operator guidance, and adding a one-click deep-link from the chat failure notice to the Claude Code guided setup card in Settings.

Changes:
- Server: rewrite Claude Code auth-required messaging and introduce a stable `(claude_code_auth_required)` marker for UI behavior.
- Client: detect the marker and render an “Open Claude Code setup” recovery button that navigates to Settings → External agents and focuses the guided setup card.
- UI polish: add a brief “focus flash” animation when deep-linking into the guided setup card; add UI tests for marker stripping + action rendering.

Reviewed changes

Copilot reviewed 10 out of 10 changed files in this pull request and generated 3 comments.

💡 <a>Add Copilot custom instructions for smarter, more guided reviews. Learn how to get started.
</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>internal/agentadapters/registry_test.go:477-479</file>
<original_commit>a3dcab0</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The comment says the UI pattern-matches this token in TranscriptMessageRow, but the actual pattern-match/rendering for the setup button happens in ChatView (TranscriptMessageRow only strips the marker from the visible text). Updating this comment to point at the real call site will prevent future contract changes from missing the correct UI handler.

</comment_old>

</comment_thread>

<comment_thread>
<file>ui/src/features/settings/SettingsView.tsx:508</file>
<original_commit>a3dcab0</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
focusTarget is read from sessionStorage and interpolated directly into a CSS selector. This can break if the stored string contains quotes/brackets and also allows selector injection (scrolling/highlighting the wrong element). Prefer using a dedicated stable id/data attribute (not data-testid) and either CSS.escape() or document.getElementById to locate the element safely.

</comment_old>

</comment_thread>

<comment_thread>
<file>ui/src/features/settings/SettingsView.tsx:511-516</file>
<original_commit>a3dcab0</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The inner timeout that removes the "settings-focus-flash" class isn’t cleared on unmount/tab switch. Consider storing its handle and clearing it in the effect cleanup to avoid running against a detached element and to prevent timer leaks if the component unmounts quickly.
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>4421467353</comment_id>
<author>@chicoxyzzy</author>
Round 2 — all three Copilot concerns addressed in `<latest commit>`:

1. Focus-target injection (`SettingsView.tsx`). Added a `KNOWN_FOCUS_TARGETS` allowlist (`Set<string>` containing only `"claude-code-guided-setup"` for now). The sessionStorage value is validated against the set before it touches a DOM query, so a malformed or unexpected token just gets ignored. Producer and consumer are both internal, so this is the right shape — extending to a new target is "add to the set + add to the producer," no escaping logic needed. Comment in the code explains how to extend.

2. Leaky inner timer (`SettingsView.tsx`). The 2.2s class-removal timeout was being created inside the outer 0ms deferred timeout but never tracked. Both handles are now stored and cleared in the effect's cleanup, so an unmount mid-flash neither leaks the timer nor runs `classList.remove` on a detached node.

3. Misleading "UI handler" pointers (`auth_status.go` + `registry_test.go`). Both comments said `TranscriptMessageRow` is where the `claude_code_auth_required` marker is matched. The actual `.includes()` call lives in `ChatView.tsx`, which builds the `setupAction` prop and passes it down. `TranscriptMessageRow` only strips the marker from the visible message text. Both comments now name `ChatView.tsx` explicitly with the file path, and call out the responsibility split so future readers don't wander to the wrong file.

Verification: `bun run typecheck` clean, `bun run test` 707/707, `go test ./internal/agentadapters/... -count=1` 213/213. No new tests this round — the existing `TranscriptMessageRow.test.tsx` coverage (marker stripping + button render + cancelled-state negation) still pins the behavior; the allowlist + timer-cleanup are internal robustness changes that don't add new observable surfaces to test.

```
  @copilot please re-review.

  </comment_new>
</pr_comments>

</comments>

----
The last **2** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address
them. Ignore comments that are not actionable or directed at
you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you
can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment
and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>




<system_reminder>
You have the capability to call multiple tools in a single
response.
For maximum efficiency, whenever you need to perform multiple
independent operations, ALWAYS call tools simultaneously
whenever the actions can be done in parallel rather than
sequentially (e.g. multiple reads/edits to different files).
Especially when exploring repository, searching, reading files,
viewing directories, validating changes. For example, you can
read 3 different files in parallel, or edit different files in
parallel. However, if some tool calls depend on previous calls
to inform dependent values like the parameters, do NOT call
these tools in parallel and instead call them sequentially (e.g.
reading shell output from a previous command should be
```

```
sequential as it requires the sessionID).
</system_reminder>

A **CI/Build Failure** has been mentioned in the problem
statement, make sure to follow your instructions for addressing
CI/Build Failures.
```